

School of Computer Science and Artificial Intelligence

Lab Assignment # 11.2

Program	: B. Tech (CSE)
Specialization	: -
Course Title	: AI Assisted Coding
Course Code	: 23CS002PC304
Semester	II
Academic Session	: 2025-2026
Name of Student	: K. Ruthwik Bhat
Enrollment No.	: 2403A51L08
Batch No.	51
Date	: 13/03/26

Submission Starts here

Screenshots:

Task Description #1 (Stack Using AI Guidance)

- Task: With the help of AI, design and implement a Stack data structure supporting basic stack operations

prompt:

Design and implement a Stack data structure in Python.

Requirements:

1. Create a class named Stack.
2. Implement the following basic stack operations:
 - push(item): Add an element to the top of the stack.
 - pop(): Remove and return the top element of the stack.
 - peek(): Return the top element without removing it.
 - is_empty(): Check if the stack is empty.
 - size(): Return the number of elements in the stack.
3. Use a Python list internally to store stack elements.
4. Add proper comments and a docstring explaining the stack and its operations.
5. Include a simple test section that demonstrates push, pop, and peek operations.
6. Print the stack contents after each operation.

```

ai_coding.py >...
3 # Task Description -1 - (Stack Using AI Guidance)
4 # • Task: With the help of AI, design and implement a Stack data
5 # structure supporting basic stack operations.
6 class Stack:
7     def __init__(self):
8         self.stack = []
9
10    def push(self, item):
11        """Add an item to the top of the stack."""
12        self.stack.append(item)
13
14    def pop(self):
15        """Remove and return the item at the top of the stack. Raises an error if the stack is empty."""
16        if not self.is_empty():
17            return self.stack.pop()
18        else:
19            raise IndexError("Stack is empty")
20
21    def peek(self):
22        """Return the item at the top of the stack without removing it. Raises an error if the stack is empty."""
23        if not self.is_empty():
24            return self.stack[-1]
25        else:
26            raise IndexError("Stack is empty")
27
28    def is_empty(self):
29        """Check if the stack is empty."""
30        return len(self.stack) == 0
31
32    def size(self):
33        """Return the number of items in the stack."""
34        return len(self.stack)
35
36    # Example usage:
37    if __name__ == "__main__":
38        stack = Stack()
39        stack.push(1)
40        stack.push(2)

```

Output:

```

36 if __name__ == "__main__":
37     stack = Stack()
38     stack.push(1)
39     stack.push(2)
40     stack.push(3)
41     print("Top item:", stack.peek()) # Output: Top item: 3
42     print("Stack size:", stack.size()) # Output: Stack size: 3
43     print("Popped item:". stack.pop()) # Output: Popped item: 3

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

PS C:\Users\Eshwar\OneDrive\Desktop\python> & "C:\Program Files\Python314\python.exe" c:/Users/Eshwar/OneDrive/Desktop/python/ai_coding.py
Top item: 3
Stack size: 3
Popped item: 3
Stack is empty: False
PS C:\Users\Eshwar\OneDrive\Desktop\python>

```

Explanation:

1. The prompt asks AI to create a **Stack data structure in Python** using a class.
2. It includes basic stack operations such as **push, pop, peek, is_empty, and size**.
3. It also requires **comments, documentation, and a test example** to demonstrate how the stack works.

Task Description #2 - (Queue Design)

- Task: Use AI assistance to create a Queue data structure

following FIFO principles

Prompt:

Design and implement a Queue data structure in Python following the FIFO (First In First Out) principle.

Requirements:

1. Create a class named Queue.
2. Implement the following operations:
 - enqueue(item): Add an element to the rear of the queue.
 - dequeue(): Remove and return the element from the front of the queue.
 - peek(): Display the front element without removing it.
 - is_empty(): Check whether the queue is empty.
 - size(): Return the number of elements in the queue.
3. Use a Python list to store queue elements.
4. Add proper comments and a docstring explaining the Queue and FIFO concept.
5. Include a small test section demonstrating enqueue and dequeue operations with sample outputs.

```
ai_coding.py 2 ...
14 class Queue:
15     """
16     A Queue is a linear data structure that follows the FIFO (First In First Out) principle.
17     This means that the first element added to the queue will be the first one to be removed.
18     The Queue class provides methods to enqueue, dequeue, peek, check if it's empty, and
19     """
20
21     def __init__(self):
22         """Initialize an empty queue."""
23         self.items = []
24
25     def enqueue(self, item):
26         """Add an element to the rear of the queue."""
27         self.items.append(item)
28
29     def dequeue(self):
30         """Remove and return the element from the front of the queue. Raises an exception
31         if self.is_empty():
32             raise IndexError("Dequeue from an empty queue")
33         return self.items.pop(0)
34
35     def peek(self):
36         """Display the front element without removing it. Raises an exception if the queue
37         is self.is_empty():
38             raise IndexError("Peek from an empty queue")
39         return self.items[0]
40
41     def is_empty(self):
42         """Check whether the queue is empty."""
43         return len(self.items) == 0
44
45     def size(self):
46         """Return the number of elements in the queue."""
47         return len(self.items)
48 # Test section demonstrating enqueue and dequeue operations with sample outputs
49 if __name__ == "__main__":
50     queue = Queue()

```

● Update is ready, click to restart.



```

ai_coding.py > ...
51
52 # Enqueue elements
53 queue.enqueue(1)
54 queue.enqueue(2)
55 queue.enqueue(3)
56
57 print("Queue after enqueueing 1, 2, 3:", queue.items) # Output: [1, 2, 3]
58
59 # Peek at the front element
60 print("Front element (peek):", queue.peek()) # Output: 1
61
62 # Dequeue elements
63 print("Dequeued element:", queue.dequeue()) # Output: 1
64 print("Queue after dequeuing:", queue.items) # Output: [2, 3]
65
66 print("Is the queue empty?", queue.is_empty()) # Output: False
67 print("Size of the queue:", queue.size()) # Output: 2
68 # Dequeue remaining elements
69 print("Dequeued element:", queue.dequeue()) # Output: 2
70 print("Dequeued element:", queue.dequeue()) # Output: 3
71 print("Is the queue empty?", queue.is_empty()) # Output: True
72 # Attempting to dequeue from an empty queue will raise an exception
73 try:
74     queue.dequeue()
75 except IndexError as e:
76     print(e) # Output: Dequeue from an empty queue
77 # Attempting to peek from an empty queue will raise an exception
78 try:
79     queue.peek()
80 except IndexError as e:
81     print(e) # Output: Peek from an empty queue
82
83

```

Output:

```

PS C:\Users\Eshwar\OneDrive\Desktop\python> & "C:\Program Files\Python314\python.exe" c:/Users/Eshwar/OneDrive/Desktop/python/ai_codin
Queue after enqueueing 1, 2, 3: [1, 2, 3]
Front element (peek): 1
Dequeued element: 1
Queue after dequeuing: [2, 3]
Is the queue empty? False
Size of the queue: 2
Dequeued element: 2
Dequeued element: 3
Is the queue empty? True
Dequeue from an empty queue
Peek from an empty queue
PS C:\Users\Eshwar\OneDrive\Desktop\python>

```

Explanation:

The task asks AI to design and implement a Queue data structure in Python.

The queue must follow the FIFO (First In First Out) principle where the first element added is the first removed.

It should include basic operations like enqueue, dequeue, peek, is_empty, and size to manage queue elements.

Task Description #3- (Singly Linked List Construction)

- Task: Utilize AI to build a singly linked list supporting insertion and traversal.

Prompt:

Design and implement a Singly Linked List in Python.

Requirements:

1. Create a Node class containing data and a reference to the next node.
2. Create a SinglyLinkedList class.
3. Implement the following operations:
 - insert(data): Insert a new node at the end of the list.
 - insert_at_beginning(data): Insert a node at the start of the list.
 - traverse(): Display all elements of the linked list.
4. Add comments and a docstring explaining how a singly linked list works.
5. Include a simple test section demonstrating insertion and traversal of nodes.

```
14 class Node:
15     """
16     A Node in a singly linked list contains data and a reference to the next node.
17     """
18     def __init__(self, data):
19         self.data = data # Store the data
20         self.next = None # Initialize the next reference to None
21
22 class SinglyLinkedList:
23     """
24     A Singly Linked List is a linear data structure where each element (node) points to the next node in the sequence.
25     It allows for efficient insertion and traversal of elements.
26     """
27     def __init__(self):
28         self.head = None # Initialize the head of the list to None
29
30     def insert(self, data):
31         """Insert a new node with the given data at the end of the list."""
32         new_node = Node(data) # Create a new node
33         if not self.head: # If the list is empty, set the new node as the head
34             self.head = new_node
35             return
36         last_node = self.head # Start from the head
37         while last_node.next: # Traverse to the end of the list
38             last_node = last_node.next
39         last_node.next = new_node # Link the last node to the new node
40
41     def insert_at_beginning(self, data):
42         """Insert a new node with the given data at the start of the list."""
43         new_node = Node(data) # Create a new node
44         new_node.next = self.head # Link the new node to the current head
45         self.head = new_node # Update the head to be the new node
46
47     def traverse(self):
48         """Display all elements of the linked list."""
49         current_node = self.head # Start from the head
50         while current_node: # Traverse until we reach the end of the list
```

Output:

```
50     while current_node: # Traverse until we reach the end of the list
51         print(current_node.data) # Print the data of the current node
52         current_node = current_node.next # Move to the next node
53 # Test section demonstrating insertion and traversal of nodes
54 if __name__ == "__main__":
55     linked_list = SinglyLinkedList() # Create a new singly linked list
56     linked_list.insert(10) # Insert nodes at the end
57     linked_list.insert(20)
58     linked_list.insert(30)
59     linked_list.insert_at_beginning(5) # Insert a node at the beginning
60     print("Elements in the linked list:")
61     linked_list.traverse() # Traverse and display the elements of the list
62
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
PS C:\Users\Eshwar\OneDrive\Desktop\python> & "C:\Program Files\Python314\python.exe" c:/Users/Eshwar/OneDrive/Desktop/python/ai_coding.py
Elements in the linked list:
5
10
20
30
PS C:\Users\Eshwar\OneDrive\Desktop\python> |
```

Explanation:

1. The prompt asks AI to **create a Singly Linked List in Python** using Node and LinkedList classes.
2. It should support **insertion of nodes and traversal** to display all elements.
3. The program should include **comments and example usage** to demonstrate how the linked list works.

Task Description -4 – (Binary Search Tree Operations)

- Task: Implement a Binary Search Tree with AI support focusing on insertion and traversal.

Prompt:

Design and implement a Binary Search Tree (BST) in Python.

Requirements:

1. Create a Node class containing data, left child, and right child.
2. Create a BinarySearchTree class.
3. Implement the following operations:
 - insert(data): Insert a new node into the BST following BST rules.
 - inorder_traversal(): Traverse the tree in inorder and display elements.

- preorder_traversal(): Traverse the tree in preorder.
 - postorder_traversal(): Traverse the tree in postorder.
4. Add comments and a docstring explaining how a Binary Search Tree works.
 5. Include a test section demonstrating insertion of multiple values and displaying the traversal outputs.

```
ai_coding.py > Node
15 class Node:
16     def __init__(self, data):
17         self.data = data
18         self.left = None
19         self.right = None
20 class BinarySearchTree:
21
22     def __init__(self):
23         self.root = None
24
25     def insert(self, data):
26         """Insert a new node with the given data into the BST."""
27         if self.root is None:
28             self.root = Node(data)
29         else:
30             self._insert_recursive(self.root, data)
31
32     def _insert_recursive(self, current_node, data):
33         if data < current_node.data:
34             if current_node.left is None:
35                 current_node.left = Node(data)
36             else:
37                 self._insert_recursive(current_node.left, data)
38         else:
39             if current_node.right is None:
40                 current_node.right = Node(data)
41             else:
42                 self._insert_recursive(current_node.right, data)
43
44     def inorder_traversal(self):
45         """Traverse the tree in inorder and return a list of elements."""
46         return self._inorder_recursive(self.root)
47
48     def _inorder_recursive(self, node):
49         elements = []
50         if node:
51             elements += self._inorder_recursive(node.left)
```

```

52         elements.append(node.data)
53         elements += self._inorder_recursive(node.right)
54     return elements
55
56     def preorder_traversal(self):
57         """Traverse the tree in preorder and return a list of elements."""
58         return self._preorder_recursive(self.root)
59
60     def _preorder_recursive(self, node):
61         elements = []
62         if node:
63             elements.append(node.data)
64             elements += self._preorder_recursive(node.left)
65             elements += self._preorder_recursive(node.right)
66         return elements
67
68     def postorder_traversal(self):
69         """Traverse the tree in postorder and return a list of elements."""
70         return self._postorder_recursive(self.root)
71
72     def _postorder_recursive(self, node):
73         elements = []
74         if node:
75             elements += self._postorder_recursive(node.left)
76             elements += self._postorder_recursive(node.right)
77             elements.append(node.data)
78         return elements
79
80     # Test section demonstrating insertion of multiple values and displaying the traversal outputs
81     if __name__ == "__main__":
82         bst = BinarySearchTree()
83         values_to_insert = [7, 3, 9, 1, 5, 8, 10]
84         for value in values_to_insert:
85             bst.insert(value)
86
87         print("Inorder Traversal:", bst.inorder_traversal())

```

Output:

```

86     print("Inorder Traversal:", bst.inorder_traversal())
87     print("Preorder Traversal:", bst.preorder_traversal())
88     print("Postorder Traversal:", bst.postorder_traversal())
89

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Eshwar\OneDrive\Desktop\python> & "C:\Program Files\Python314\python.exe" c:/Users/Eshwar/
Inorder Traversal: [1, 3, 5, 7, 8, 9, 10]
Preorder Traversal: [7, 3, 1, 5, 9, 8, 10]
Postorder Traversal: [1, 5, 3, 8, 10, 9, 7]
PS C:\Users\Eshwar\OneDrive\Desktop\python>

```

Explanation:

1. The prompt asks AI to create a Binary Search Tree (BST) in Python using Node and BinarySearchTree classes.

2. It focuses on inserting elements according to BST rules and performing tree traversals (inorder, preorder, postorder).
3. The program should include comments and example tests to show how the BST operations work.

Task Description -5 – (Hash Table Implementation) •

Task: Create a hash table using AI with collision handling

Prompt:

Design and implement a Hash Table in Python.

Requirements:

1. Create a HashTable class.
2. Implement a hash function to convert keys into index values.
3. Handle collisions using a method such as chaining (linked lists) or linear probing.
4. Implement the following operations:
 - insert(key, value): Add a key–value pair to the hash table.
 - search(key): Retrieve the value associated with the key.
 - delete(key): Remove a key–value pair from the table.
5. Add comments and a docstring explaining hashing and collision handling.
6. Include a test section demonstrating insertion, searching, and deletion operations.

```

ai_coding.py > ...
15 class HashTable:
16     def __init__(self, size=10):
17         """Initialize the hash table with a specified size."""
18         self.size = size
19         self.table = [[] for _ in range(size)] # Create a list of empty lists for chaining
20
21     def hash_function(self, key):
22         """Compute the hash value for a given key."""
23         return hash(key) % self.size
24
25     def insert(self, key, value):
26         """Add a key-value pair to the hash table."""
27         index = self.hash_function(key)
28         # Check if the key already exists and update it
29         for i, (k, v) in enumerate(self.table[index]):
30             if k == key:
31                 self.table[index][i] = (key, value) # Update existing key
32                 return
33         # If the key does not exist, add it to the chain
34         self.table[index].append((key, value))
35
36     def search(self, key):
37         """Retrieve the value associated with the key."""
38         index = self.hash_function(key)
39         for k, v in self.table[index]:
40             if k == key:
41                 return v # Return the value if found
42         return None # Return None if the key is not found
43
44     def delete(self, key):
45         """Remove a key-value pair from the table."""
46         index = self.hash_function(key)
47         for i, (k, v) in enumerate(self.table[index]):
48             if k == key:
49                 del self.table[index][i] # Remove the key-value pair
50                 return True # Return True if deletion was successful

```

Output:

```

51         return False # Return False if the key was not found
52 # Test section demonstrating insertion, searching, and deletion operations
53 if __name__ == "__main__":
54     # Create a hash table
55     hash_table = HashTable()
56
57     # Insert key-value pairs
58     hash_table.insert("name", "Alice")
59     hash_table.insert("age", 30)
60     hash_table.insert("city", "New York")
61
62     # Search for values
63     print(hash_table.search("name")) # Output: Alice
64     print(hash_table.search("age")) # Output: 30
65     print(hash_table.search("city")) # Output: New York
66     print(hash_table.search("country")) # Output: None (not found)
67
68     # Delete a key-value pair
69     print(hash_table.delete("age")) # Output: True (deletion successful)
70     print(hash_table.search("age")) # Output: None (not found after deletion)
71     print(hash_table.delete("age")) # Output: False (already deleted)

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS  Pyt

```

PS C:\Users\Eshwar\OneDrive\Desktop\python> & "C:\Program Files\Python314\python.exe" c:/Users/Eshwar/OneDrive/Desktop/python/ai_coding.py
Alice
30
New York
None
True
None
False
False
PS C:\Users\Eshwar\OneDrive\Desktop\python>

```

Expllination:

1. The prompt asks AI to create a Hash Table in Python for storing key–value pairs.
2. It must include a hash function and collision handling method such as chaining or linear probing.
3. The program should support insert, search, and delete operations with comments and example tests.